OBJECT ORIENTED PROGRAMMING
UNIT - III

AI303PC : OBJECT ORIENTED PROGRAMMING USING JAVA

CONCEPTS OF EXCEPTION HANDLING

Exceptions

- An exception is an abnormal condition that arises when executing a program.
- In the languages that do not support exception handling, errors must be checked and handled manually, usually through the use of error codes.
- In contrast, Java:

  1) provides syntactic mechanisms to signal, detect and handle errors

  2) ensures a clean separation between the code executed in the absence of errors and the code to handle various kinds of errors

  3) brings run-time error management into object-oriented programming

EXCEPTION HANDLING

- An exception is an object that describes an exceptional condition (error) that has occurred when executing a program.
- Exception handling involves the following:

1)When an error occurs, an object (exception) representing this error is created and thrown in the method that caused it.

2) That method may choose to handle the exception itself or pass it on.

3) Either way, at some point, the exception is caught and processed.

EXCEPTION SOURCES

- **Exceptions can be:**

  1)Generated by the Java run-time system. Fundamental errors that violate the rules of the Java language or the constraints of the Java execution environment.

  2) Manually generated by a programmer's code. Such exceptions are typically used to report some error conditions to the caller of a method.

EXCEPTION CONSTRUCTS

- **Five constructs are used in exception handling:**
  1) **try** – a block surrounding program statements to monitor for exceptions
  2) **catch** – together with try, catches specific kinds of exceptions and handles them in some way
  3) **finally** – specifies any code that absolutely must be executed whether or not an exception occurs
  4) **throw** – used to throw a specific exception from the program
  5) **throws** – specifies which exceptions a given method can throw

EXCEPTION-HANDLING BLOCK

General form:

where:

1) try { … } is the block of code to monitor for exceptions
2) catch(Exception ex) { … } is exception handler for the exception "Exception"
3) finally { … } is the block of code to execute before the try block ends

BENEFITS OF EXCEPTION HANDLING

- Separating Error-Handling code from "regular" business logic code.
- Propagating errors up the call stack.
- Grouping and differentiating error types.

SEPARATING ERROR HANDLING CODE FROM REGULAR CODE

- In traditional programming, error detection, reporting, and handling often lead to confusing code.
- Consider the pseudocode method here that reads an entire file into memory.

```
readFile {
  open the file;
  determine its size;
  allocate that much memory;
  read the file into memory;
  close the file;
}
```

TRADITIONAL PROGRAMMING: NO SEPARATION OF ERROR-HANDLING CODE

- In traditional programming, to handle such cases, the *readFile* method must have more code to do error detection, reporting, and handling.
- The following **slide#10** contains the code of the *readFile* method.

TRADITIONAL PROGRAMMING: NO SEPARATION OF ERROR-HANDLING CODE

```
errorCodeType readFile{
  initialize errorCode = 0;
  open the file;
  if (theFileIsOpen){
    determine the length of the file;
    if (gotTheFileLength) {
        allocate that much memory;
        if (gotEnoughMemory) {
            read the file into memory;
            if (readFailed){
                errorCode = -1;
            }else{
                errorCode = -2;
            }
        }else{
            errorCode = -3;
        }
        close the file;
        if(theFileDidntClose && errorCode == 0){
            errorCode = -4;
        }else{
            errorCode = errorCode and -4;
        }
    }else{
        errorCode = -5;
    }
  }
    return errorCode;
}
```

SEPARATING ERROR HANDLING CODE FROM REGULAR CODE

- Exceptions enable you to write the main flow of your code and to deal with the exceptional cases elsewhere.

- Note that exceptions don't spare you the effort of doing the work of detecting, reporting, and handling errors, but they do help you organize the work more effectively.
- Code is provided in the following **slide#12**.

SEPARATING ERROR HANDLING CODE FROM REGULAR CODE

```
readFile{
  try{
    open the file;
    determine its size;
    allocate that much memory;
    read the file into memory;
    close the file;
  }catch(fileOpenFailed) {
    doSomething;
  }
```

```
  catch(sizeDeterminationFailed) {
    doSomething;
  }catch(memoryAllocationFailed) {
    doSomething;
  }catch(readFailed) {
    doSomething;
  }catch(fileCloseFailed) {
    doSomething;
  }
}
```

PROPAGATING ERRORS UP THE CALL STACK

- Suppose that the *readFile* method is the fourth method in a series of nested method calls made by the main program: method1 calls *method2*, which calls *method3*, which finally calls *readFile.*
- Suppose also that method1 is the only method interested in the errors that might occur within *readFile*.

```
method1 {
    call method2;
}
method2 {
    call method3;
}
method3 {
    call readFile;
}
```

TRADITIONAL WAY OF PROPAGATING ERRORS

- Traditional error notification, Techniques force method2 and method3 to propagate the error codes returned by readFile up the call stack until the error codes finally reach method1 - the only method that is interested in them.

```
method1 {
  errorCodeType error;
  error = call method2;
  if (error)
    doErrorProcessing;
  else
    proceed;
}
```

```
errorCodeType method2 {
  errorCodeType error;
  error = call method3;
  if (error)
    return error;
  else
    proceed;
}
```

```
errorCodeType method3 {
  errorCodeType error;
  error = call readFile;
  if (error)
    return error;
  else
    proceed;
}
```

USING JAVA EXCEPTION HANDLING

- Any checked exceptions that can be thrown within a method must be specified in its throws clause.

```
method1 {
  try {
    call method2;
  } catch (exception e) {
    doErrorProcessing;
  }
}
method2 throws exception {
    call method3;
}
method3 throws exception {
    call readFile;
}
```

GROUPING AND DIFFERENTIATING ERROR TYPES

- Because all exceptions thrown within a program are objects, the grouping or categorizing of exceptions is a natural outcome of the class hierarchy
- An example of a group of related exception classes in the Java platform are those defined in **java.io.IOException** and its descendants
- **IOException** is the most general and represents any type of error that can occur when performing I/O
- Its descendants represent more specific errors. For example, **FileNotFoundException** means that a file could not be located on disk.

GROUPING AND DIFFERENTIATING ERROR TYPES

- A method can write specific handlers that can handle a very specific exception.
- The **FileNotFoundException** class has no descendants, so the following handler can handle only one type of exception.

```
catch (FileNotFoundException e) {
    ........
}
```

GROUPING AND DIFFERENTIATING ERROR TYPES

- A method can catch an exception based on its group or general type by specifying any of the exception's superclasses in the catch statement.
- For example, to catch all I/O exceptions, regardless of their specific type, an exception handler specifies an **IOException** argument.

```
// Catch all I/O exceptions, including
// FileNotFoundException, EOFException, and so on.
catch (IOException e) {
    ......
}
```

TERMINATION VS. RESUMPTION

- There are two basic models in exception-handling theory.
- In *termination,* the error is so critical that there's no way to get back to where the exception occurred. Whoever threw the exception decided that there was no way to salvage the situation, and they don't *want* to come back.
- The alternative is called *resumption*. It means that the exception handler is expected to do something to rectify the situation, and then the faulting method is retried, presuming success the second time. If you want resumption, it means you still hope to continue execution after the exception is handled.

TERMINATION VS. RESUMPTION

- In *resumption*, a method call expects resumption-like behavior, meaning that when an exceptional condition occurs, the method does not throw the exception. Instead, it fixes the problem internally and then resumes execution from the point of failure.
- A common way to simulate this behavior is to place the try block inside a while loop, allowing the code to repeatedly reenter the try block until a valid or satisfactory result is obtained.
- In practice, however, operating systems that originally supported resumptive exception handling eventually shifted toward termination-like approaches, effectively skipping true resumption because termination proved simpler, safer, and easier to maintain.

EXCEPTION HIERARCHY

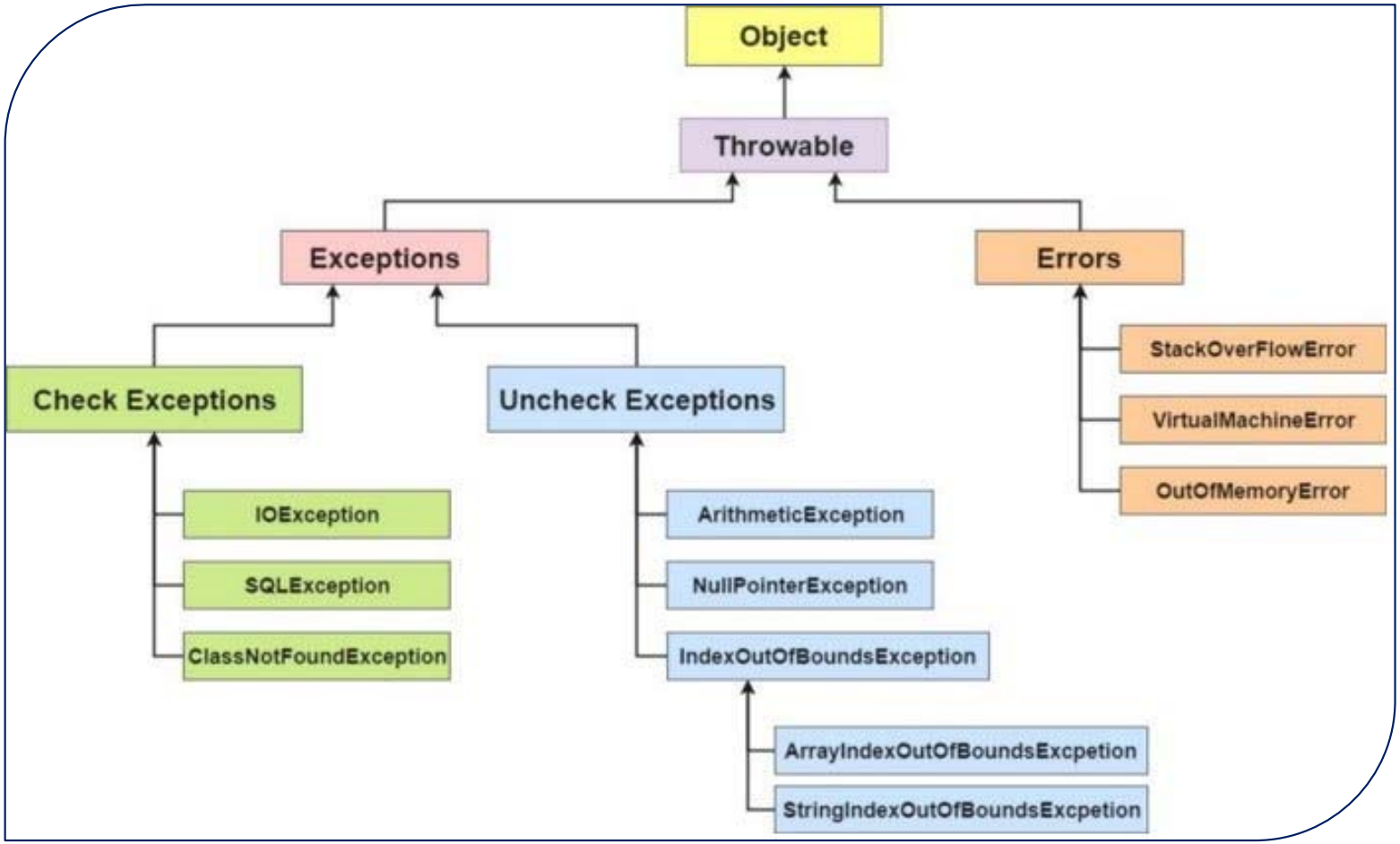
- All exceptions are subclasses of the built-in class Throwable.
- Throwable contains two immediate subclasses:
  1. Exception – exceptional conditions that programs should catch.

     The class includes:

     a) **RuntimeException** – defined automatically for user programs to include: division by zero, invalid array indexing, etc.

     b) use-defined exception classes.
  2. Error – exceptions used by Java to indicate errors with the runtime environment; user programs are not supposed to catch them.

HIERARCHY OF EXCEPTION CLASSES



USAGE OF TRY-CATCH STATEMENTS

- Syntax:

```
try {
    <code to be monitored for exceptions>
}catch (<ExceptionType1> <ObjName>) {
    <handler if ExceptionType1 occurs>
} ...

} catch (<ExceptionTypeN> <ObjName>) {
    <handler if ExceptionTypeN occurs>
}
```

CATCHING EXCEPTIONS: THE TRY-CATCH STATEMENTS

```
class DivByZero {
  public static void main(String args[]) {
  try {
    System.out.println(3/0);
    System.out.println("Please print me.");
  }catch (ArithmeticException exc) {
   //Division by zero is an ArithmeticException
   System.out.println(exc);
  }
   System.out.println("After exception.");
  }
}
```

CATCHING EXCEPTIONS: MULTIPLE CATCH

```
class MultipleCatch{
  public static void main(String args[]){
    try{
      int den = Integer.parseInt(args[0]);
      System.out.println(3/den);
    }catch (ArithmeticException exc){
       System.out.println("Divisor was 0.");
    }catch (ArrayIndexOutOfBoundsException exc2){
       System.out.println("Missing argument.");
    }
    System.out.println("After exception.");
  }
}
```

CATCHING EXCEPTIONS: NESTED TRY'S

```
class NestedTryDemo{
  public static void main(String args[]){
    try {
      int a = Integer.parseInt(args[0]);
      try {
        int b = Integer.parseInt(args[1]);
        System.out.println(a/b);
      }catch (ArithmeticException e){
        System.out.println("Div by zero error!");
      }
    }catch (ArrayIndexOutOfBoundsException){
        System.out.println("Need 2 parameters!");
    }
  }
}
```

CATCHING EXCEPTIONS: NESTED TRY'S WITH METHODS

```
class NestedTryDemo2 {
   static void nestedTry(String args[]) {
     try {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        System.out.println(a/b);
       }catch (ArithmeticException e) {
            System.out.println("Div by zero error!");
       }
   }
   public static void main(String args[]){
     try {
       nestedTry(args);
     }catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Need 2 parameters!");
     }
   }
}
```

THROWING EXCEPTIONS(THROW)

- So far, we have only caught the exceptions thrown by the Java system.
- In fact, a user program may throw an exception explicitly:

  throw ThrowableInstance;

- **ThrowableInstance** must be an object of type Throwable or its subclass.

THROWING EXCEPTIONS(THROW)

- Once an exception is thrown by:

  throw **ThrowableInstance**;

  1. The flow of control stops immediately
  2. The nearest enclosing try statement is inspected if it has a catch statement that matches the type of exception:
     1. If one exists, control is transferred to that statement.
     2. Otherwise, the next enclosing try statement is examined.
     3. If no enclosing try statement has a corresponding catch clause, the default exception handler halts the program and prints the stack.

CREATING EXCEPTIONS

- Two ways to obtain a Throwable instance:
  1. Creating one with the new operator
     - All Java built-in exceptions have at least two Constructors:
     - One without parameters and another with one String parameter:
       - throw new NullPointerException( );
       - throw new NullPointerException("demo");
  2. Using a parameter of the catch clause.
     - try { … } catch(Throwable e) { … e … }

EXAMPLE: THROW 1

```
class ThrowDemo {
/*The method demoproc throws a NullPointerException
exception which is immediately caught in the try block
and re-thrown:*/
static void demoproc() {
  try {
    throw new NullPointerException("demo");
  } catch(NullPointerException e) {
    System.out.println("Caught inside demoproc.");
    throw e;
  }
}
```

EXAMPLE: THROW 2

- The main method calls demoproc within the try block, which catches and handles the NullPointerException.
- **exception:**

```
public static void main(String args[]){
  try {
    demoproc();
  } catch(NullPointerException e) {
    System.out.println("Recaught: " + e);
  }
}
```

THROWS DECLARATION

- If a method is capable of causing an exception that it does not handle, it must specify this behavior by the throws clause in its declaration:

```
type name(parameter-list) throws exception-list{
    ......
}
```

- where exception-list is a comma-separated list of all types of exceptions that a method might throw.
- All exceptions must be listed except Error and **RuntimeException** or any of their subclasses; otherwise, a compile-time error occurs.

EXAMPLE: THROWS 1

- The throwOne( ) method throws an exception that it does not catch, nor declares it within the throws clause.

```
class ThrowsDemo{
  static void throwOne( ){
    System.out.println("Inside throwOne.");
    throw new IllegalAccessException("demo");
  }

  public static void main(String args[]){
    throwOne( );
  }
}
```

- Therefore, this program does not compile.

EXAMPLE: THROWS 2

- **Corrected program**: throwOne lists exception, main catches it:

```
class ThrowsDemo {
  static void throwOne() throws IllegalAccessException {
      System.out.println("Inside throwOne.");
      throw new IllegalAccessException("demo");
  }
  public static void main(String args[]) {
    try {
        throwOne();
    } catch (IllegalAccessException e) {
        System.out.println("Caught " + e);
    }
  }
}
```

FINALLY

- When an exception is thrown:
  1. The execution of a method is changed
  2. The method may even return prematurely.
- This may be a problem in many situations.
- For instance, if a method opens an entry file and closes on exit, exception handling should not bypass the proper closure of the file.
- The finally block is used to address this problem.

FINALLY, CLAUSE

- The try/catch statement requires at least one catch or finally clause, although both are optional:

```
try {
    ...
}catch(Exception1 ex1){
    ...
}finally{
    ...
}
```

- Executed after try/catch whether of not the exception is thrown.
- Any time a method is to return to a caller from inside the try/catch block via:
  1. uncaught exception or
  2. explicit return
- The finally clause is executed just before the method returns.

EXAMPLE: FINALLY, 1

- Three methods to exit in various ways.

```
class FinallyDemo {
/*procA prematurely breaks out of the try by
throwing an exception, the finally clause is executed
on the way out:*/
static void procA() {
  try {
      System.out.println("inside procA");
      throw new RuntimeException("demo");
  }finally {
      System.out.println("procA's finally");
  }
}
```

EXAMPLE: FINALLY, 2

```
/* procB's try statement is exited via a return statement,
the finally clause is executed before procB returns:*/

static void procB() {
  try {
    System.out.println("inside procB");
    return;
  } finally {
    System.out.println("procB's finally");
  }
}
```

EXAMPLE: FINALLY, 3

- In procC, the try statement executes normally without error, however the finally clause is still executed:

```
static void procC() {
  try {
    System.out.println("inside procC");
  } finally {
    System.out.println("procC's finally");
  }
}
```

EXAMPLE: FINALLY, 4

- Demonstration of the three methods:

```
public static void main(String args[]) {
  try {
    procA();
  } catch (Exception e) {
    System.out.println("Exception caught");
  }
  procB();
  procC();
  }
}
```

JAVA BUILT-IN EXCEPTIONS

- The default **java.lang** package provides several exception classes, all subclassing the **RuntimeException** class.
- Two sets of built-in exception classes:
  1. unchecked exceptions – the compiler does not check if a method handles or throws the exceptions
  2. checked exceptions – must be included in the method's throws clause if the method generates but does not handle them

UNCHECKED BUILT-IN EXCEPTIONS

- Methods that generate but do not handle those exceptions need not declare them in the throws clause:
    1. ArithmeticException
    2. ArrayIndexOutOfBoundsException
    3. ArrayStoreException
    4. ClassCastException
    5. IllegalStateException
    6. IllegalMonitorStateException
    7. IllegalArgumentException

UNCHECKED BUILT-IN EXCEPTIONS

8. StringIndexOutOfBounds
9. UnsupportedOperationException
10. SecurityException
11. NumberFormatException
12. NullPointerException
13. NegativeArraySizeException
14. IndexOutOfBoundsException
15. IllegalThreadStateException

CHECKED BUILT-IN EXCEPTIONS

- Methods that generate but do not handle those exceptions must declare them in the throws clause:
    1. NoSuchMethodException
    2. NoSuchFieldException
    3. InterruptedException
    4. InstantiationException
    5. IllegalAccessException
    6. CloneNotSupportedException
    7. ClassNotFoundException

CREATING OWN EXCEPTION CLASSES

- Built-in exception classes handle some generic errors.
- For application-specific errors, define your own exception classes. How? Define a subclass of Exception:

  class MyException extends Exception { … }
- MyException need not implement anything – its mere existence in the type system allows us to use its objects as exceptions.

EXAMPLE: OWN EXCEPTIONS 1

- A new exception class is defined, with a private detail variable, a one-parameter constructor and an overridden toString method:

```
class MyException extends Exception {
  private int detail;
  MyException(int a) {
    detail = a;
  }
  public String toString() {
    return "MyException[" + detail + "]";
  }
}
```

EXAMPLE: OWN EXCEPTIONS 2

```
class ExceptionDemo {
  /* The static compute method throws the MyException
  exception whenever its a argument is greater than 10:*/

  static void compute(int a) throws MyException {
    System.out.println("Called compute(" + a + ")");
    if (a > 10) throw new MyException(a);
      System.out.println("Normal exit");
  }
  // Code continues in next slide#49
```

EXAMPLE: OWN EXCEPTIONS 3

- The main method calls compute with two arguments within a try block that catches the **MyException** exception:

```
// Code continuation from Slide#48
  public static void main(String args[]) {
    try {
        compute(1);
        compute(20);
    } catch (MyException e) {
        System.out.println("Caught " + e);
    }
  }
}
```

MULTITHREADING VS MULTITASKING

Multitasking (OS-level)

- **Definition:** Running multiple processes concurrently on a computer. Each process has its own address space and OS-level resources.
- **Types:** Preemptive multitasking (OS scheduler interrupts processes) and cooperative (processes yield).
- **Isolation:** Strong - memory/protected; inter-process communication (IPC) required.

MULTITHREADING VS MULTITASKING

Multithreading (within a process)

- **Definition:** Multiple threads of execution inside the same process. Threads share memory and resources (heap, file descriptors), but each thread has its own call stack and program counter.
- **Why use threads:** concurrency for I/O-bound tasks, parallelism on multi-core CPUs, responsiveness in GUIs and servers.
- **Risks:** race conditions, visibility problems, deadlocks, and resource starvation.

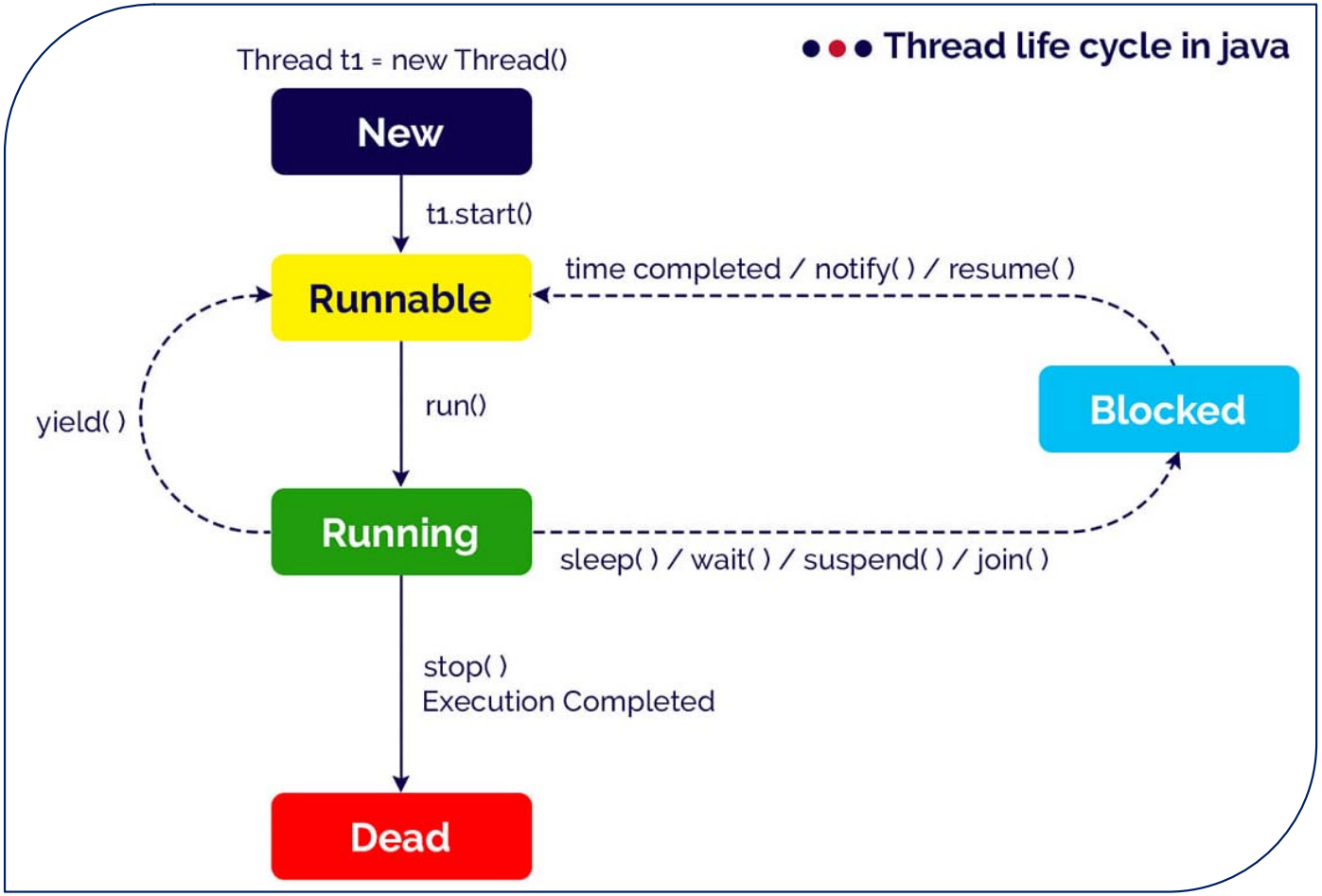
MULTITHREADING VS MULTITASKING

Feature	Multitasking	Multithreading
Unit of execution	Process	Thread
Memory usage	High	Low
Context switch time	High	Low
Sharing of data	Not shared	Shared
Performance	Slower	Faster
Example	Running Chrome + VS Code	Multiple tabs loading in Chrome

WHAT ARE THREADS?

- Threads are sometimes called lightweight processes. Both processes and threads provide an execution environment, but creating a new thread requires fewer resources than creating a new process.
- Threads exist within a process - every process has at least one. Threads share the process's resources, including memory and open files. This makes for efficient, but potentially problematic, communication.
- Multithreaded execution is an essential feature of the Java platform. Every application has at least one thread — or several, if you count "system" threads that do things like memory management and signal handling. But from the application programmer's point of view, you start with just one thread, called the main thread.

THREAD LIFE CYCLE



THREAD STATES

- States (Java Thread.State): NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- NEW: created (new Thread(...)) but not yet start()-ed.
- RUNNABLE: eligible to run. JVM maps this to OS threads (implementation-dependent). A RUNNABLE thread may be actually running or waiting for the CPU.
- BLOCKED: waiting to enter a synchronized block because another thread holds the monitor.
- WAITING: called Object.wait() or Thread.join() (without timeout) — waiting indefinitely until notified or the joined thread finishes.
- TIMED_WAITING: sleep(millis), wait(timeout), join(timeout) — will wake after timeout if not earlier.
- TERMINATED: run() returned or thread died due to an exception.

CREATING THREADS

In Java, there are two ways to create and start a new thread.

1. **Using the Runnable interface**:

- You create a class that implements Runnable and puts the thread's code in the run() method.This Runnable object is then passed to the Thread constructor.
- Advantage: This is the more flexible and preferred method because your class can still extend another class (Java only allows single inheritance). This separates the task (Runnable) from the thread (Thread).
- Note: Using Runnable is the more general and recommended approach.

CREATING THREADS (WAY-1 EXAMPLE)

```java
public class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable on " + Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable(), "R-Thread");
        t.start();
    }
}
```

CREATING THREADS

2. Subclass the Thread class:

- You create a class that extends Thread and overrides the run() method with your code.
- Limitation: Your class cannot extend any other class.In both cases, you must call the start() method on the Thread object to begin execution.
- Note: You must call start(), not run(). Calling start() tells the Java Virtual Machine (JVM) to create a new execution thread and call the run() method in that new thread. Calling run() directly just executes the method in the current thread like any normal method call.

CREATING THREADS (WAY-2 EXAMPLE)

```java
public class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Hello from " + Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.setName("T1");
        t.start(); // DO NOT call run() directly
    }
}
```

THREAD PRIORITIES

- Thread priority in Java helps manage execution order in multithreaded applications, influencing how tasks compete for CPU time. While higher-priority threads may execute first, scheduling depends on the OS and JVM behavior.
- Practically, thread priority is useful for real-time processing, background tasks, and resource-intensive operations, ensuring critical tasks get CPU time when needed.

THREAD PRIORITIES

- Java has three default priority levels:
  1. Thread.MIN_PRIORITY (value 1)
  2. Thread.NORM_PRIORITY (value 5)
  3. Thread.MAX_PRIORITY (value 10)
- The default priority for a thread is Thread.NORM_PRIORITY, meaning unless explicitly set, threads are treated equally.
- In many cases, relying solely on thread priorities may not be sufficient, as the operating system could override these priorities.
- For example, on some systems, thread priority in Java might not be strictly enforced, or the OS may allocate CPU time based on its own criteria. Therefore, thread priority should be used in conjunction with other concurrency control mechanisms (like synchronization) to achieve optimal performance in multithreaded applications.

THREAD PRIORITIES (EXAMPLE - CODE)

```
class ThreadPriority {
    public static void main(String... args) {
        Runnable task = new Runnable() {
            public void run() {
                String name = Thread.currentThread().getName();
                for (int i = 0; i < 1000; i++) {
                    if (i % 200 == 0)
                        System.out.println(name + " is running. Count: " + i);
                }
            }
        };
        Thread high = new Thread(task, "HighThread");
        high.setPriority(Thread.MAX_PRIORITY); // Priority 10
        Thread low = new Thread(task, "LowThread");
        low.setPriority(Thread.MIN_PRIORITY); // Priority 1
        System.out.println("Starting threads with priorities 10 and 1...");
        high.start();   low.start();
    }
}
```

SYNCHRONIZING THREADS

- Threads communicate primarily by sharing access to fields and the objects reference fields refer to.
- This form of communication is extremely efficient, but makes two kinds of errors possible: thread interference and memory consistency errors.
- The tool needed to prevent these errors is synchronization.

Why synchronize?

- To avoid race conditions on shared mutable state and to ensure visibility (one thread seeing another thread's writes).

SYNCHRONIZING THREADS

Java primitives

1. synchronized keyword (method or block)
2. volatile variables
3. Higher-level: java.util.concurrent.locks.Lock (e.g., ReentrantLock)
4. Atomic classes: AtomicInteger, AtomicReference, etc.
5. Concurrent collections: ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue.

Synchronized (monitor)

- Acquires intrinsic lock (monitor) for object/class.

SYNCHRONIZING THREADS (EXAMPLE CODE)

```
class ThreadSync {
    private int count = 0; // Shared resource
    public void increment() {
        synchronized (this) {
            count++;
        }
    }
    public static void main(String[] args) throws Exception {
        ThreadSync shared = new ThreadSync();
        Runnable task = new Runnable() {
          // CODE CONTINUES IN SLIDE#66
```

SYNCHRONIZING THREADS (EXAMPLE CODE)

```
        // CONTINUATION OF CODE FROM SLIDE#65
            @Override
            public void run() {
                for (int i = 0; i < 100000; i++) shared.increment();
            }
        };
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); // Wait for t1 to finish
        t2.join(); // Wait for t2 to finish
        System.out.println("Final Count: " + shared.count);
    }
}
```

INTER-THREAD COMMUNICATION

Why Threads Need to Communicate

- The main problem threads face is coordination. If you have a Producer thread creating data and a Consumer thread processing it, they must coordinate:
  1. The Consumer must wait if the data store is empty.
  2. The Producer must wait if the data store is full.
  3. The Producer must tell the Consumer when new data is ready.
  4. The Consumer must tell the Producer when it has finished processing data.
- Without communication, the threads would either waste CPU time constantly checking the status (polling) or, worse, process data that isn't ready, leading to errors.

INTER-THREAD COMMUNICATION (MECHANISM)

- In Java, the simplest and most fundamental way for threads to communicate is through the methods inherited from the Object class: wait(), notify(), and notifyAll().
- wait(): Causes the current thread to pause execution and release the object's lock, entering the waiting state until notified or interrupted.
- notify(): Wakes up a single thread that is waiting on the object's lock, allowing it to compete for the lock once the notifier releases it.
- notifyAll(): Wakes up all threads that are waiting on the object's lock, allowing all of them to compete for the lock once the notifier releases it.

INTER-THREAD COMMUNICATION (PRODUCER – CONSUMER EXAMPLE)

```java
import java.util.concurrent.TimeUnit; // For clean sleep
// 1. Shared Resource and Communication Hub
class Items {
    int count = 1000;
    // Consumer logic: Must use 'while' and 'wait()'
    public synchronized void consume(int cnt) throws InterruptedException {
        System.out.println("C: Need " + cnt + ", Current: " + this.count);
        while (this.count < cnt) { // Must re-check condition
            System.out.println("C: Low, waiting.");
            wait();
        }
        this.count -= cnt;
        System.out.println("C: Consumed. New: " + this.count);
    }
    // Producer logic
    public synchronized void produce(int cnt) throws InterruptedException {
        System.out.println("P: Producing " + cnt);
        this.count += cnt;
        System.out.println("P: Produced. New: " + this.count);
        notifyAll(); // Signal all waiting consumers
    }
}
```

INTER-THREAD COMMUNICATION (PRODUCER – CONSUMER EXAMPLE)

```java
// 2. Main Class to run the demo
public class InterThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        final Items obj = new Items();
        Thread consumer = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    obj.consume(1500);
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }
        }, "Consumer");
        Thread producer = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    TimeUnit.MILLISECONDS.sleep(100);
                    obj.produce(800);
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }
        }, "Producer");
        consumer.start();       producer.start();
        consumer.join();        producer.join();
    }
}
```

THREAD GROUPS

- Thread groups in Java provide a mechanism for managing and organizing threads into a single hierarchical structure. Think of them as folders 📂 for threads, allowing you to treat a collection of threads as a single unit.
- The main purpose of a thread group is to set properties for a group of threads all at once and manage their overall status.
- A Java thread group is a hierarchical tree structure that organizes and manages threads (like a folder) to allow bulk control (e.g., setting setMaxPriority or interrupting all threads) and provides default assignment for new threads, while historically also defining security boundaries.

THREAD GROUPS (EXAMPLE CODE)

```java
public class ThreadGroupDemo {
    public static void main(String[] args) {
        // 1. Create a ThreadGroup
        ThreadGroup salesGroup = new ThreadGroup("Sales Processing");
        System.out.println("Group Name: " + salesGroup.getName());
        // 2. Create threads and assign them to the group
        Runnable task = () -> System.out.println(
            Thread.currentThread().getName() +
            " running in group: " +
            Thread.currentThread().getThreadGroup().getName()
        );
        Thread t1 = new Thread(salesGroup, task, "Sales Agent 1");
        Thread t2 = new Thread(salesGroup, task, "Sales Agent 2");
        // 3. Start threads
        t1.start();     t2.start();
        // 4. Perform group operation (e.g., getting the active count)
        System.out.println("Active threads in group: " + salesGroup.activeCount());
        // 5. Set a property for the whole group
        salesGroup.setMaxPriority(Thread.NORM_PRIORITY);
}
```

DEAMON THREADS

- Definition: background threads that do not prevent JVM exit. When only daemon threads remain, JVM exits.
- Use-cases: background monitoring, housekeeping tasks. Avoid using for important I/O because JVM can exit before daemon finishes.
- Must call setDaemon(true) before start().

DEAMON THREADS (EXAMPLE CODE)

```
class DaemonThreadDemo {
    public static void main(String args[]){
        Thread t=new Thread(new Runnable(){
            @Override
            public void run() {
                System.out.println("Daemon thread started.");
            }
        });
        t.setDaemon(true);
        t.start();
        System.out.println("End of main thread");
    }
}
```

ENUMERATIONS (ENUM)

- Definition: special class type used to define a finite set of constant values.
- Syntax:
  - public enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }
- To create an enum, use the enum keyword (instead of class or interface), and separate the constants with a comma. Note that they should be in uppercase letters as provided in the Syntax above.
- You can access enum constants with the **dot.**
- Syntax:
  - Day myVar = Day.MON;

DIFFERENCE B/W ENUM AND CLASS

- An enum can, just like a class, have attributes and methods. The only difference is that enum constants are public, static and final (unchangeable - cannot be overridden).
- An enum cannot be used to create objects, and it cannot extend other classes (but it can implement interfaces).

Why And When To Use Enums?

- Use enums when you have values that you know aren't going to change, like month days, days, colors, deck of cards, etc.

ENUMERATION (EXAMPLE CODE - 1)

```java
public class Main {
  enum Level {
    LOW,
    MEDIUM,
    HIGH
  }
  public static void main(String[] args) {
    Level myVar = Level.MEDIUM;
    System.out.println(myVar);
  }
}
```

ENUMERATION (EXAMPLE CODE - 2)

```
public class EnumDeo {
  enum Level {
    LOW,
    MEDIUM,
    HIGH
  }
  public static void main(String[] args) {
    for (Level myVar : Level.values()) {
        System.out.println(myVar);
    }
  }
}
```

AUTOBOXING & UNBOXING

- Autoboxing: automatic conversion from primitive to wrapper class (e.g., int -> Integer).
- Unboxing: wrapper -> primitive.
- Examples:
  - Integer i = 5; // autoboxing
  - int j = i; // unboxing
- Limitations:
  - NullPointerException: Integer i = null; int j = i; -> NPE at unboxing.
  - Identity vs equality: Integer a = 127, b = 127; a==b may be true because of caching; Integer x = 128; Integer y = 128; x==y is false. Use .equals() for value equality.

AUTOBOXING & UNBOXING (EXAMPLE CODE)

```java
public class BoxUnboxDemo {
    public static void main(String[] args) {
        int primitiveInt = 50;
        Integer wrapperInt = primitiveInt;
        System.out.println("Autoboxing: (int) -> (Integer)");
        System.out.println("Wrapper Value: " + wrapperInt);
        int primitiveUnboxed = wrapperInt;
        System.out.println("\nUnboxing: (Integer) -> (int)");
        System.out.println("Primitive Value: " + primitiveUnboxed);
        if (primitiveInt == wrapperInt) {
            System.out.println("\nCombined: Autoboxing and Unboxing.");
            System.out.println("The comparison required unboxing.");
        }
    }
}
```

ANNOTATIONS (INTRODUCED IN JAVA SE 5)

- Definition: metadata attached to program elements – classes, methods, fields, parameters, local variables, etc.

Common built-in annotations:

1. @Override — indicates a method overrides a superclass method.
2. @Deprecated — marks deprecated API.
3. @SuppressWarnings("unchecked") — suppresses compiler warnings.

Meta-annotations (annotations on annotations):

1. @Retention(RetentionPolicy.RUNTIME|CLASS|SOURCE) - when annotation retained.
2. @Target(ElementType.TYPE|METHOD|FIELD|PARAMETER|...) - applicable elements. @Documented, @Inherited, @Repeatable.

ANNOTATIONS (CUSTOM ANNOTATION EXAMPLE CODE)

```java
import java.lang.annotation.*;

@Documented
@Target(ElementType.METHOD)
@Inherited
@Retention(RetentionPolicy.RUNTIME)
public @interface MethodInfo{
    String author() default "KHAN";
    String date();
    int revision() default 1;
    String comments();
}
```

ANNOTATIONS (CUSTOM ANNOTATION EXAMPLE CODE)

```java
import java.lang.annotation.*;
@Documented
@Target(ElementType.METHOD)
@Inherited
@Retention(RetentionPolicy.RUNTIME)
public @interface MethodInfo{
    String author() default "Shafakhat";
    int revision() default 1;
}
```

ANNOTATIONS (BUILT-IN ANNOTATIONS EXAMPLE CODE)

```java
import java.io.FileNotFoundException;
import java.util.*;
public class AnnotationExample {
    public static void main(String[] args) { ..... }
    @Override
    @MethodInfo(author = "Shafakhat", revision = 1)
    public String toString() {
        return "Overriden toString method";
    }
    @Deprecated
    @MethodInfo(author = "Shafakhat")
    public static void oldMethod() {
        System.out.println("old method, don't use it.");
    }
    @SuppressWarnings({ "unchecked", "deprecation" })
    @MethodInfo(revision = 10)
    public static void genericsTest() throws FileNotFoundException {
        List l = new ArrayList();
        l.add("abc");
        oldMethod();
    }
}
```

GENERICS

- Java Generics allows us to create a single class, interface, and method that can be used with different types of data (objects). This helps us to reuse our code.
- Note: Generics does not work with primitive types (int, float, char, etc).

Advantages of Generics:

1. Code Reusability: Generics allow writing methods and classes that operate on different types of data without needing to rewrite the code (e.g., public <T> void genericsMethod(T data)).
2. Compile-time Type Checking: Specifying a type parameter (e.g., GenericsClass<Integer>) ensures that the code works only with the specified data type, catching type-mismatch errors before runtime.
3. Used with Collections: Generics enable the Collections Framework (like ArrayList, Map, Queue) to handle different data types while maintaining type safety.

GENERICS

Java Generic Class

- A generic class is a class that uses a type parameter so it can work with different data types while providing compile-time type safety.

Java Generic Method

- A generic method is a method that declares its own type parameter(s) allowing it to operate on any data type independently of the class's type.

Java Generic Interface

- A generic interface is an interface defined with a type parameter so implementing classes can specify the concrete data type they will use.

Java Bounded Types

- A bounded type restricts a generic type parameter to accept only types that either extend a specific class or implement a specific interface.

GENERICS (EXAMPLE CODE)

```
class Box<T>{
    T v; Box(T v){this.v=v;}
    <U> void show(U u){System.out.println(v+" "+u);}
    <N extends Number> void num(N n){System.out.println(n);}
}
public class Demo{
    public static void main(String[] a){
        Box<String> b = new Box<>("Hi");
        b.show(99);
        b.num(5);
    }
}
```